

Java Constructors

A beginner-friendly guide with real examples — Syeda Ajbina Nusrat, Lecturer, CSE, UITS

■ Start Here: Think of it Like a House Blueprint

Imagine a **class** is a **blueprint for a house**. Every time you build a new house from that blueprint, you need to set it up — give it an address, decide the number of rooms, paint the walls. That setup process is exactly what a **constructor** does for an object.

When you write **new House()** in Java, the constructor runs automatically and prepares the object for use — you don't call it manually, Java calls it for you.

1. What is a Constructor?

A constructor is a **special block of code** that runs automatically the moment an object is created. Its job is to **initialize the object's fields** (give them starting values).

Rule	Details
Same name as class	If your class is 'Car', the constructor must be named 'Car'
No return type	Not even void — constructors never return anything
Called automatically	Java calls it when you write 'new ClassName()'
Can be overloaded	You can have multiple constructors with different parameters
Default provided	If you write no constructor, Java provides an empty one automatically

■ **Remember:** A constructor is NOT a method — it's a special initializer. The key giveaway: **no return type at all** (not even void).

2. Types of Constructors

2a. Default Constructor (No-Argument)

Takes no parameters. Sets all fields to **default starting values**. If you don't write any constructor, Java automatically creates one that does nothing.

Example — a simple Dog class:

```
class Dog {
    String name;
    String breed;

    // Default constructor
    public Dog() {
        name = "Unknown";
        breed = "Mixed";
    }
}
```

```

    }
}

// Creating a dog object
Dog d = new Dog(); // constructor runs automatically
// d.name = "Unknown", d.breed = "Mixed"

```

2b. Parameterized Constructor

Takes parameters so you can **set custom values at the time of creation**. This way, each object can start with different data.

Example — two different students with different data:

```

class Student {
    int id;
    String name;
    double gpa;

    // Parameterized constructor
    public Student(int id, String name, double gpa) {
        this.id = id;          // 'this' refers to the current object
        this.name = name;
        this.gpa = gpa;
    }
}

Student s1 = new Student(101, "Alice", 3.8);
Student s2 = new Student(102, "Bob", 3.2);
// s1 and s2 are completely independent objects with their own data

```

■ **What is 'this'?** When a parameter has the same name as a field, **this.name** refers to the object's field, while **name** alone refers to the parameter. Always use 'this.' inside constructors to avoid confusion.

3. Constructor Overloading

A class can have **multiple constructors** as long as each has a different parameter list (different number or types of parameters). Java picks the right one based on what arguments you pass.

Real-world analogy: ordering a pizza. You can order with just a size, or with a size + topping, or with size + topping + extra cheese. Same pizza shop, different order forms — that's constructor overloading.

Example — Circle class with 3 constructors:

```

class Circle {
    double x, y, radius;

    // Constructor 1: full details
    public Circle(double x, double y, double radius) {

```

```

        this.x = x; this.y = y; this.radius = radius;
    }

    // Constructor 2: only radius, centre defaults to (0,0)
    public Circle(double radius) {
        this.x = 0; this.y = 0; this.radius = radius;
    }

    // Constructor 3: no args, unit circle at origin
    public Circle() {
        this.x = 0; this.y = 0; this.radius = 1.0;
    }
}

Circle c1 = new Circle(10, 12, 20); // centre(10,12), radius=20
Circle c2 = new Circle(5); // centre(0,0), radius=5
Circle c3 = new Circle(); // centre(0,0), radius=1

```

Call	Constructor Used	Result
new Circle(10, 12, 20)	Constructor 1 (3 params)	Centre=(10,12), r=20
new Circle(5)	Constructor 2 (1 param)	Centre=(0,0), r=5
new Circle()	Constructor 3 (0 params)	Centre=(0,0), r=1

4. Copying Objects

Java has **no built-in copy constructor** (unlike C++), but you can still duplicate an object's data in three ways:

Method	How it works	Best for
Via constructor	Pass existing object as parameter, manually copy fields	Simple classes
Direct field assignment	Create new empty object, then copy fields one by one	Quick copies, small classes
clone() method	Override Object.clone() for a built-in deep copy	Complex/large classes

Example — copy via constructor:

```

class Student {
    int id; String name;

    // Regular constructor
    Student(int id, String name) { this.id=id; this.name=name; }

    // Copy constructor – takes another Student object
    Student(Student other) {
        this.id = other.id;
        this.name = other.name;
    }
}

```

```
}  
}
```

```
Student s1 = new Student(101, "Alice");  
Student s2 = new Student(s1); // s2 is an independent copy of s1  
s2.name = "Copy of Alice"; // changing s2 does NOT affect s1
```

■ **Important:** Don't confuse copying with **reference assignment**. Writing **Student s2 = s1** does NOT copy — both s1 and s2 point to the *same* object. Any change to s2 also changes s1!

5. Constructor vs Method — The Key Difference

This is one of the most commonly confused topics for beginners. Here's the clearest way to think about it:

■ **Constructor** = setting up a brand new house (runs ONCE when object is created)

■ **Method** = doing things inside the house later (runs whenever you call it)

Aspect	Constructor	Method
Purpose	Initialize an object's state	Define an object's behaviour
Return type	NONE — not even void	Must have one (void, int, String, etc.)
Name	Must match class name exactly	Any name (camelCase convention)
When called	Automatically when 'new' is used	Explicitly by calling objectName.method()
How many times	Once per object creation	As many times as needed
Default	Java provides empty one if none exists	Java never provides one automatically
Can be called again?	No	Yes, anytime

Side-by-side code comparison:

```
class BankAccount {  
    String owner;  
    double balance;  
  
    // ■ CONSTRUCTOR – no return type, same name as class  
    public BankAccount(String owner, double initialDeposit) {  
        this.owner = owner;  
        this.balance = initialDeposit; // sets up the account  
    }  
  
    // ■ METHOD – has return type, different name  
    public void deposit(double amount) {  
        balance += amount; // does something to the object  
    }  
  
    // ■ METHOD – returns a value
```

```

    public double getBalance() {
        return balance;
    }
}

// Usage:
BankAccount acc = new BankAccount("Alice", 500.0); // constructor runs
acc.deposit(200.0); // method called manually
System.out.println(acc.getBalance()); // Output: 700.0

```

6. Common Beginner Mistakes

■ Mistake: Adding a return type to a constructor

```

public void Student() { } // ■ WRONG – this is now a regular method, NOT a constructor!
public Student() { } // ■ CORRECT

```

■ Mistake: Wrong name (doesn't match class)

```

class Car { public vehicle() { } } // ■ WRONG – constructor name must be 'Car'
class Car { public Car() { } } // ■ CORRECT

```

■ Mistake: Confusing reference copy with value copy

```

Student s2 = s1; // ■ Both point to same object – not a copy!
Student s2 = new Student(s1); // ■ Actual independent copy via copy constructor

```

■ Mistake: Forgetting 'this' when param names match field names

```

public Student(String name) { name = name; } // ■ Assigns param to itself!
public Student(String name) { this.name = name; } // ■ Assigns param to field

```

7. Full Worked Example — Put It All Together

Let's build a complete class using everything we've learned:

```

class Phone {
    String brand;
    String model;
    int batteryLevel;

    // Default constructor
    public Phone() {
        brand = "Generic"; model = "Unknown"; batteryLevel = 100;
    }

    // Parameterized constructor
    public Phone(String brand, String model) {
        this.brand = brand; this.model = model; this.batteryLevel = 100;
    }
}

```

```

// Parameterized constructor with battery level
public Phone(String brand, String model, int battery) {
    this.brand = brand; this.model = model; this.batteryLevel = battery;
}

// METHOD — uses battery
public void makeCall(int minutes) {
    batteryLevel -= minutes * 2;
    System.out.println("Called for " + minutes + " min. Battery: " + batteryLevel + "%");
}

// METHOD — returns info
public String getInfo() {
    return brand + " " + model + " (" + batteryLevel + "% battery)";
}
}

// Main usage:
Phone p1 = new Phone(); // Generic Unknown, 100%
Phone p2 = new Phone("Samsung", "S24"); // Samsung S24, 100%
Phone p3 = new Phone("Apple", "iPhone 15", 45); // Apple iPhone 15, 45%

p2.makeCall(10); // Output: Called for 10 min. Battery: 80%
System.out.println(p3.getInfo()); // Output: Apple iPhone 15 (45% battery)

```

Quick Reference Card

Question	Answer
Does a constructor have a return type?	No — never, not even void
What is the constructor name?	Must be exactly the same as the class name
When does it run?	Automatically when 'new ClassName()' is written
Can a class have multiple constructors?	Yes — as long as they have different parameters
What if I write no constructor?	Java auto-provides an empty default constructor
What does 'this' do inside a constructor?	Refers to the current object's fields/methods
Is Student s2 = s1 a copy?	No — both point to the same object in memory
How to truly copy an object?	Use a copy constructor or manual field assignment